Determinism

Algoritm determinist:

fiecare acțiune/operație are un rezultat unic determinat
serialitate: pentru orice moment de timp t din cursul execuției exista o singură acțiune efectuată la momentul t

Algoritm nedeterminist:
există acțiuni/operații al căror rezultat nu este unic definit ci are valori într-o mulțime **finită** de posibilități
paralelism: structura arborescentă de operații; operațiile de pe o cale din arbore sunt efectuate serial; operațiile de pe căi diferite sunt efectuate în paralel; la un moment de timp t se execută mai multe acțiuni pe diverse cai
nu are implementare practica

Operații caracteristice algoritmilor nedeterministi

choice(A) – ramifică copia curentă a algoritmului în cardinal(A) copii
A = mulțime **finita!**
pentru fiecare valoare din A - copie a algoritmului care continuă cu acea valoare
variabilele locale ale algoritmului sunt clonate pentru fiecare copie
copiile continuă în paralel și independent una de alta

fail – copia curentă se termină cu insucces; restul copiilor continua execuția
success – copia curentă se termina cu succes; celelalte copii sunt terminate
(practic execuția întregului algoritm se termina cu succes)

Obs1: valoarea de return a algoritmului este success (când una din cai se termină cu success) sau fail (când toate căile se termina cu fail) => algoritmii nedeterminiști de decizie sunt funcționali.

Obs2: algoritmii de optim pot fi modelați ca apeluri succesive ale unor **algoritmi de decizie** (de exemplu determinarea arborelui de acoperire minim pe un arbore fără costuri pe muchii: exista arbore de acoperire din 1 muchie? dacă nu, exista din 2 muchii? etc)

Complexitatea temporala a unui algoritm nedeterminist (s.n. complexitate angelica)

suma complexităților operațiilor din secventa/calea cea mai **scurta** care termina algoritmul cu **success**
dacă toate căile întorc fail, complexitatea este suma complexităților de pe calea cea mai lungă încheiată cu fail

Obs: choice(A) are complexitate O(1).

Etape in execuția unui algoritm nedeterminist
generare (choice – generează cate o copie pentru fiecare candidat la a fi soluție)
testare (fiecare candidat generat este testat dacă e o soluție corecta)
Obs: partea nedeterminista a algoritmului corespunde etapei de generare.

Exemple de algoritmi nedeterminiști

Cautarea unui element intr-un vector

```
// V = vectorul, n = nr de elemente din vector, e = elementul cautat
caut(V, n, e) {
i = choice(1..n)  //generare - pozitia elementului cautat
if (V[i]=e) success  //testare - este elementul cautat?
fail
}
```

Test daca un numar natural este neprim

```
// n = numarul testat
neprim(n) {
i = choice(2..floor(sqrt(n)))    // generare – posibilii divizori
if (n mod i=0) success     // testare – este i divizor pentru n?
fail
}
```

Sortarea unui vector de elemente strict pozitive

```
// V = vectorul, n = nr de elemente din vector
sort(V, n) {
	for i=1 to n
		Aux[i]=0 // in Aux vom crea vectorul sortat
	for i=1 to n {
		j = choice(1..n) // generare - poziția ocupata de V[i]
		if (Aux[j]>0) fail // testare - nu e deja ocupata?
		Aux[j] = V[i]
	}
	for i=1 to (n-1)
		if (Aux[i]>Aux[i+1]) // testare - e sortat?
			fail
	success // daca niciun test nu a dat fail, success
}
```

Tractabilitate

P = PTIME = clasa problemelor rezolvabile prin algoritmi determiniști polinomiali
NP = NPTIME = clasa problemelor rezolvabile prin algoritmi nedeterminiști polinomiali

Obs: NP nu înseamnă ca nu este P! NP vine de la “non deterministic polynomial time”, nu de la “not P”. In fapt, $P \subseteq NP$ (orice algoritm determinist polinomial poate fi ușor transformat intr-un algoritm nedeterminist polinomial), iar dacă $P = NP$ rămâne în continuare o problemă deschisa (cel mai plauzibil este că nu sunt egale, însă nu s-a putut demonstra încă).

Problema tractabila: rezolvabilă printr-un algoritm determinist polinomial

Problema intractabilă: toți algoritmii determiniști care o rezolva sunt supra-polinomiali

Reducerea polinomiala

A = algoritm pentru problema PA (pe care o reduc) B = algoritm pentru problema PB (la care reduc)
F = algoritm **determinist polinomial** de transformare a intrărilor pentru A în intrări pentru B
Dacă exista F algoritm determinist cu complexitate polinomială care transformă **orice** instanță i a problemei PA într-**o** instanță F(i) a problemei PB a.î. **A(i)=1 <=> B(F(i))=1**, atunci spunem ca PA se reduce polinomial la PB: **PA $\leq_p$ PB**
Obs0: diferența dintre reducerea polinomială și reducerea Turing este că aici F trebuie să fie determinist polinomial.
Obs1: faptul ca reducerea nu merge (neapărat) si de la PB la PA se vede în sublinierea cuvintelor "orice" și "o". Când reducem PA la PB luăm o intrare oarecare pentru PA și **construim** în mod convenabil o intrare pentru PB.
Obs2: demonstrația faptului ca o problemă se reduce polinomial la alta nu e completă dacă nu demonstrați **implicația în ambele sensuri**! Întâi trebuie demonstrat ca A(i)=1 => B(F(i))=1, apoi ca B(F(i))=1 => A(i)=1.

Consecinte ale stabilirii unei relatii de reducere polinomiala:

$A \leq_p B \wedge B \in P \Rightarrow A \in P$
$A \leq_p B \wedge A \notin P \Rightarrow B \notin P$
Aceste consecințe se folosesc pentru a demonstra apartenenta unor probleme la o anumită clasa de probleme, folosind reduceri (de) la probleme a căror (in)tractabilitate este cunoscută.

Clase de probleme

P = { A | $\exists$ un algoritm determinist polinomial care rezolvă A }
NP = { A | $\exists$ un algoritm nedeterminist polinomial care rezolvă A }
NP-hard = { A | $\forall$ Q $\in$ NP • Q $\leq_p$ A }
NP-complete = { A | A$\in$NP $\wedge$ A$\in$NP-hard }

Strategii de a demonstra apartenența la o clasă de probleme

P: se construiește un algoritm determinist polinomial
NP: se construiește un algoritm nedeterminist polinomial SAU se arata ca soluția poate fi verificată în timp polinomial
NP-hard: se găsește o problema cunoscuta ca NP-hard și se reduce aceasta problema la problema curenta (nu invers!)
NP-complete: se arată ca e NP si NP-hard

Alte clase de probleme
PSPACE = clasa problemelor care sunt rezolvate de algoritmi deterministi in timp nelimitat si utilizand spatiu polinomial
NPSPACE = clasa problemelor care sunt rezolvate de algoritmi NEdeterministi in timp nelimitat si utilizand spatiu polinomial
PSPACE = NPSPACE (Savitch, 1970)

Găsiți un algoritm nedeterminist pentru următoarele probleme:

1. k-clica:
Dându-se un graf și un număr k, exista un subgraf complet (o clica) de dimensiune k?
2. k-acoperire:
Dându-se un graf și un număr k, exista o submulțime de k noduri astfel încât fiecare muchie (v1,v2) sa aibă cel puțin unul dintre nodurile care o compun (v1 sau v2) în submulțimea aleasa?
3. problema comis-voiajorului:
Se dă o mulțime de orașe conectate intre ele prin drumuri. Exista vreo modalitate ca un comis voiajor sa viziteze toate orașele o singura data și să se întoarcă de unde a plecat?
4. Colorarea unui graf:
Dându-se un graf și 3 culori, se pot colora nodurile grafului doar cu cele 3 culori astfel încât niciun nod sa nu aibă un vecin de aceeași culoare?
5. SAT(Boolean Satisfiability Problem)
Se dă o expresie booleană în forma normala conjunctivă - o conjuncție de clauze, unde clauzele sunt disjuncții. Exemplu $(x1 \vee \neg x2) \wedge (\neg x1 \vee x2 \vee x3) \wedge \neg x1$. Să se determine dacă există o posibilitate de atribuire a variabilelor astfel încât expresia să fie adevărată.
6. Problema subgrafurilor izomorfe
Dându-se două grafuri, G și H, există un subgraf în G care să fie izomorf cu H?
7. Submulțime de sumă dată
Se da o mulțime de N numere și un numar Q. Exista o submulțime de numere a căror suma să fie fix Q?
8. Independent set
Dându-se un graf și un număr k, există o mulțime de k noduri astfel încât niciunul dintre noduri sa nu fie conectat de celelalte din mulțime?
9. Problema partiționării
Dându-se o mulțime de numere S, există două submulțimi S1 și S2 de sume egale?